

Arithmetic Functions

Dr. Muhammad Usman
Professor
Department of Computer Science
UET Mardan

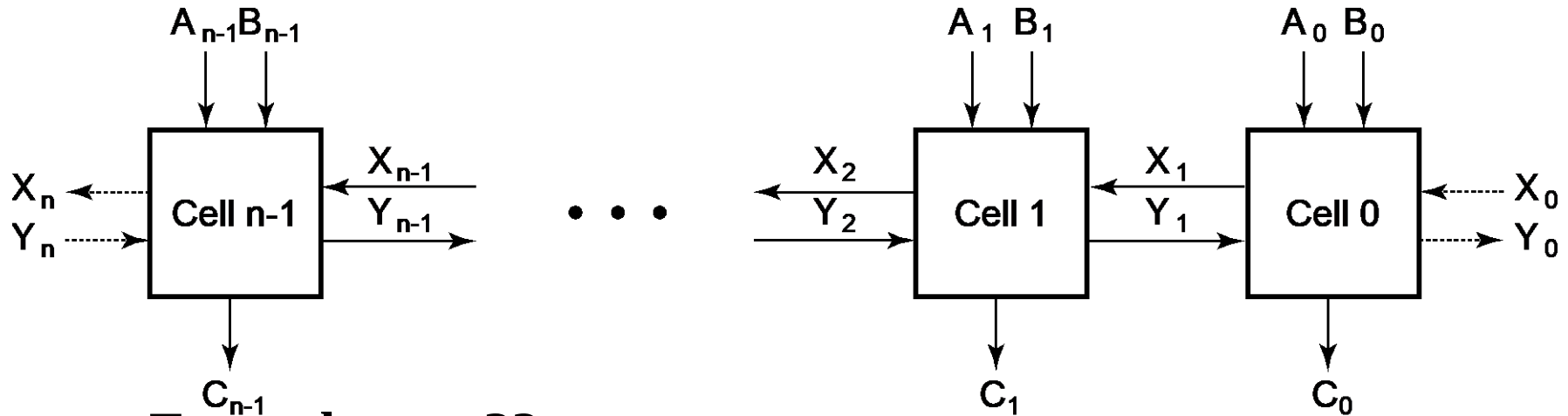
Overview

- **Iterative combinational circuits**
- **Binary adders**
 - Half and full adders
 - Ripple carry
- **Binary subtraction**
- **Binary adder-subtractors**
 - Signed binary numbers
 - Signed binary addition and subtraction
 - Overflow
- **Other arithmetic functions**
 - Design by contraction

Iterative Combinational Circuits

- **Arithmetic functions**
 - Operate on binary vectors
 - Use the same subfunction in each bit position
- **Can design functional block for subfunction and repeat to obtain functional block for overall function**
- ***Cell* - subfunction block**
- ***Iterative array* - a array of interconnected cells**
- **An iterative array can be in a single dimension (1D) or multiple dimensions**

Block Diagram of a 1D Iterative Array



- **Example: $n = 32$**
 - Number of inputs = ?
 - Truth table rows = ?
 - Equations with up to ? input variables
 - Equations with huge number of terms
 - Design impractical!
- Iterative array takes advantage of the regularity to make design feasible

Functional Blocks: Addition

- **Binary addition used frequently**
- **Addition Development:**
 - *Half-Adder (HA)*, a 2-input bit-wise addition functional block,
 - *Full-Adder (FA)*, a 3-input bit-wise addition functional block,
 - *Ripple Carry Adder*, an iterative array to perform binary addition, and
 - *Carry-Look-Ahead Adder (CLA)*, a hierarchical structure to improve performance.

Functional Block: Half-Adder

- A 2-input, 1-bit width binary adder that performs the following computations:

X	0	0	1	1
+ Y	+ 0	+ 1	+ 0	+ 1
C S	0 0	0 1	0 1	1 0

- A half adder adds two bits to produce a two-bit sum
- The sum is expressed as a sum bit, S and a carry bit, C
- The half adder can be specified as a truth table for S and C $\Rightarrow$

X	Y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Logic Simplification: Half-Adder

- The K-Map for S, C is:
- This is a pretty trivial map!
By inspection:

$$S = X \cdot \overline{Y} + \overline{X} \cdot Y = X \oplus Y$$

$$S = (X + Y) \cdot (\overline{X + Y})$$

- and

$$C = X \cdot Y$$

$$C = \overline{\overline{(X \cdot Y)}}$$

- These equations lead to several implementations.

S		C	
	Y		Y
	0	0	1
X	1 ₂	X	1 ₃

Five Implementations: Half-Adder

- We can derive following sets of equations for a half-adder:

$$\begin{array}{ll} \text{(a)} \quad S = X \cdot \bar{Y} + \bar{X} \cdot Y & \text{(d)} \quad \underline{S} = (\underline{X} + \underline{Y}) \cdot \bar{C} \\ \quad \quad C = X \cdot Y & \quad \quad \bar{C} = (\bar{X} + \bar{Y}) \end{array}$$

$$\begin{array}{ll} \text{(b)} \quad S = (X + Y) \cdot (\bar{X} + \bar{Y}) & \text{(e)} \quad S = X \oplus Y \\ \quad \quad C = \underline{X \cdot Y} & \quad \quad C = X \cdot Y \end{array}$$

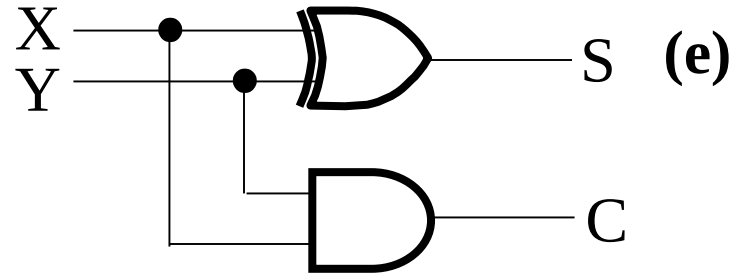
$$\begin{array}{l} \text{(c)} \quad S = (\underline{C} + \bar{X} \cdot \bar{Y}) \\ \quad \quad C = X \cdot Y \end{array}$$

- (a), (b), and (e) are SOP, POS, and XOR implementations for S.
- In (c), the C function is used as a term in the AND-NOR implementation of S, and in (d), the $\bar{C}$ function is used in a POS term for S.

Implementations: Half-Adder

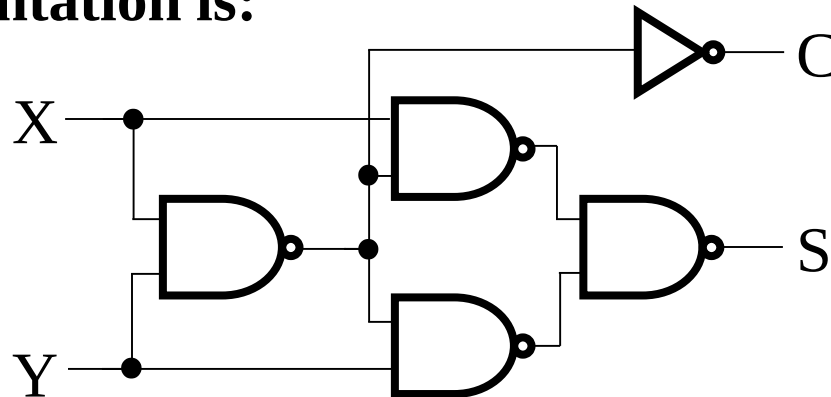
- The most common half adder implementation is:

$$S = X \oplus Y$$
$$C = X \cdot Y$$



- A NAND only implementation is:

$$S = (X + Y) \cdot C'$$
$$C = ((X \cdot Y))$$



Functional Block: Full-Adder

- A full adder is similar to a half adder, but includes a carry-in bit from lower stages. Like the half-adder, it computes a sum bit, S and a carry bit, C.

- For a carry-in (Z) of 0, it is the same as the half-adder:

Z	0	0	0	0
X	0	0	1	1
<u>+ Y</u>	<u>+ 0</u>	<u>+ 1</u>	<u>+ 0</u>	<u>+ 1</u>
C S	0 0	0 1	0 1	1 0

- For a carry- in (Z) of 1:

Z	1	1	1	1
X	0	0	1	1
<u>+ Y</u>	<u>+ 0</u>	<u>+ 1</u>	<u>+ 0</u>	<u>+ 1</u>
C S	0 1	1 0	1 0	1 1

Logic Optimization: Full-Adder

- Full-Adder Truth Table:

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Full-Adder K-Map:

K-Map for Sum (S):

			Y
	0	1	3
	1		2
X	1		
	4	5	7
			6
		Z	

K-Map for Carry (C):

			Y
	0	1	3
	1		2
X		1	
	4	5	7
			6
		Z	

$$S = \overline{X}\overline{Y}Z + \overline{X}Y\overline{Z} + \overline{X}YZ + X\overline{Y}\overline{Z}$$

$$C = XY + XZ + YZ$$

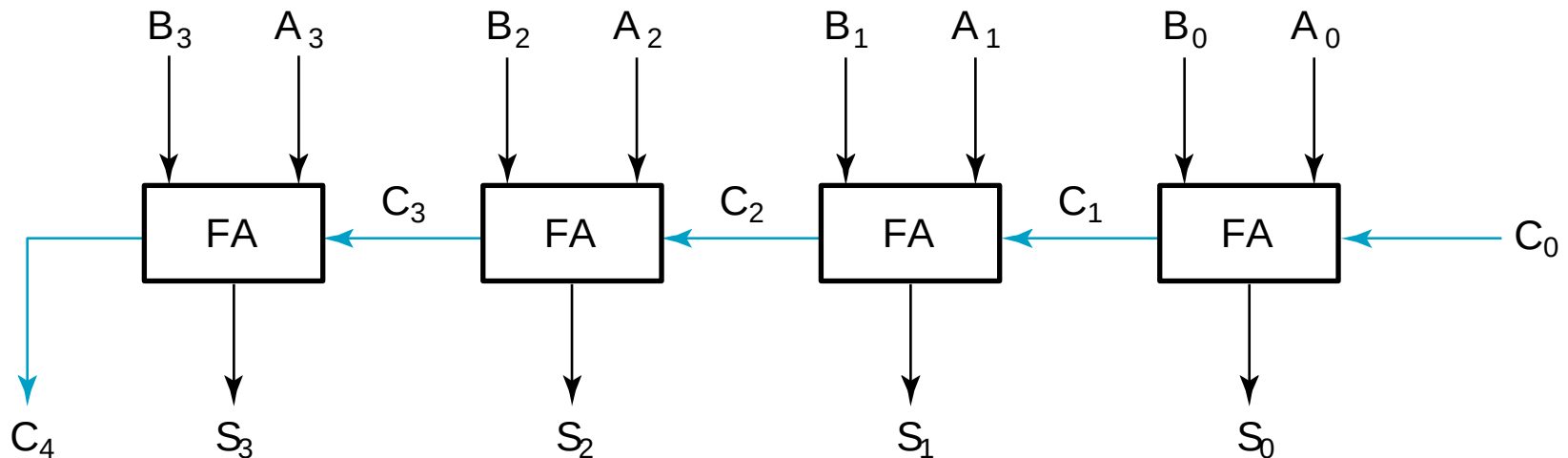
Binary Adders

- To add multiple operands, we “bundle” logical signals together into vectors and use functional blocks that operate on the vectors
- Example: 4-bit ripple carry adder: Adds input vectors $A(3:0)$ and $B(3:0)$ to get a sum vector $S(3:0)$
- Note: carry out of cell i becomes carry in of cell $i + 1$

Description	Subscript 3 2 1 0	Name
Carry In	0 1 1 0	C_i
Augend	1 0 1 1	A_i
Addend	<u>0 0 1 1</u>	B_i
Sum	1 1 1 0	S_i
Carry out	0 0 1 1	C_{i+1}

4-bit Ripple-Carry Binary Adder

- A four-bit Ripple Carry Adder made from four 1-bit Full Adders:



Multi-bit Addition

Addition of two binary numbers in parallel implies that all the bits of augend and addend are available for computation at the same time.

- Total propagation time is equal to the propagation delay of a typical gate times the number of gate levels in the circuit.
- The longest propagation delay time in a parallel adder is the time it takes the carry to propagate through the full adders.
- Each bit of the sum output depends on the value of the input carry.

■ The value of S_i in any given stage in the adder will be in its steady state final value only after the input carry to that stage has been propagated.

- Inputs A_4 and B_4 reach a steady value as soon as input signals are applied to the adder.
- But input carry C_4 does not settle to its final steady state value until C_3 is available in its steady state value.
- C_3 has to wait for C_2 and so on down to C_1 .

If there are four full adders in the parallel adder, the output carry C_5 would have $2 \times 4 = 8$ gate levels from C_1 to C_5 .

- The total propagation time in the adder would be the propagation time in one half adder plus eight gate level.
- For an n bit parallel adder there are $2n$ gate level for the carry to propagate through.
- The carry propagation is a limiting factor on the speed with which two numbers are added in parallel.

The solution is to employ faster gates with reduced delays. But this is a limitation.

- There are several techniques for reducing the carry propagation time in a parallel adder.
- The most widely used technique is the principle of look-ahead carry.

Equations: Full-Adder

- The S function is the three-bit XOR function (Odd Function):

$$S = X \oplus Y \oplus Z$$

- The Carry bit C is 1 if both X and Y are 1 (the sum is 2), or if the sum is 1 and a carry-in (Z) occurs. Thus C can be re-written as:

$$C = X Y + (X \oplus Y) Z$$

- The term $X \cdot Y$ is *carry generate*.
- The term $X \oplus Y$ is *carry propagate*.

■ Consider the circuit of the full adder, if we define two new binary variables, ■

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

The output sum and carry can be expressed as,

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

G_i is called a carry generate and it produces an output carry when both A_i and B_i are one regardless of the input carry.

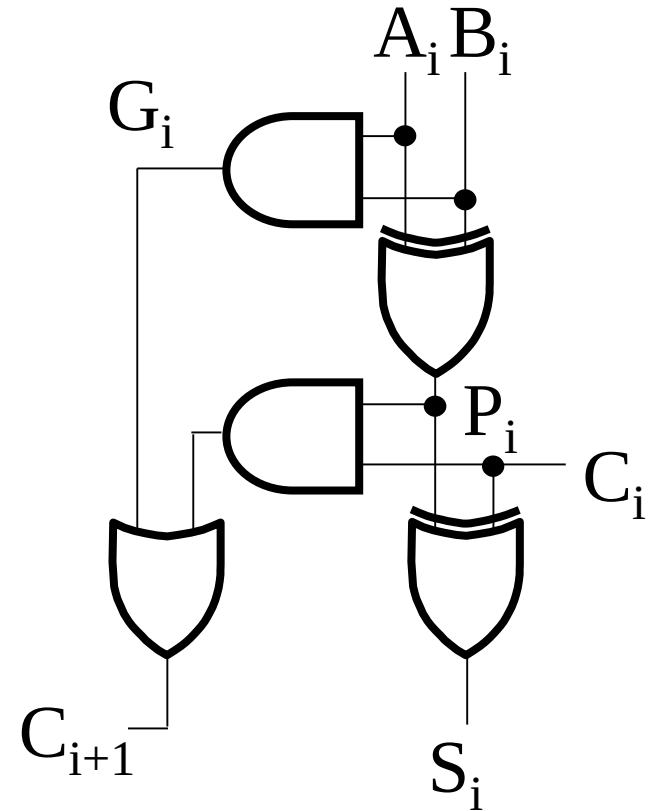
- P_i is called carry propagate because it is the term associated with the propagation of the carry from C_i to C_{i+1} .

Implementation: Full Adder

- **Full Adder Schematic**
- Here X, Y, and Z, and C (from the previous pages) are A, B, C_i and C_o , respectively. Also,
 G = generate and
 P = propagate.
- **Note:** This is really a combination of a 3-bit odd function (for S)) and Carry logic (for C_o):

(G = Generate) OR (P = Propagate AND C_i = Carry In)

$$C_o = G + P \cdot C_i$$



The inputs and output variables use the subscript i to denote a typical stage in the parallel adder.

- The signals at P_i and G_i settle to their steady state values after the propagation through their respective gates.
- These two signals are common to all full adders and depend only on the input augend and addend bits.
- The signal from the input carry C_i to the output carry C_{i+1} , propagates through an AND gate and an OR Gate, which constitute two gate level.

- The Boolean function for the carry output of each stage and substitute for each C_i , its value from the previous equation.

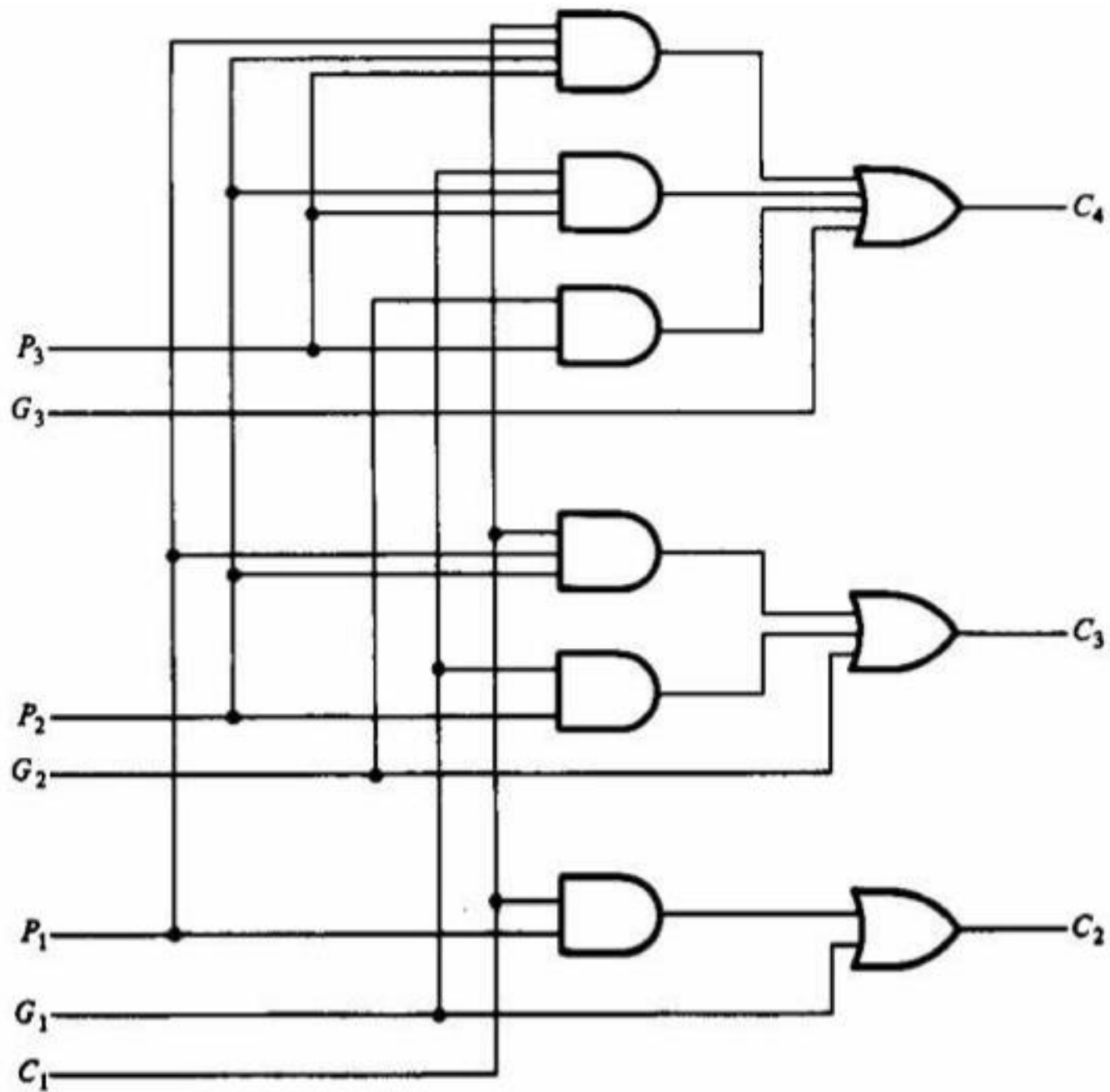
$$C_2 = G_1 + P_1 C_1$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 C_1) = G_2 + P_2 G_1 + P_2 P_1 C_1$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 C_1$$

Each output carry is expressed in SOP.

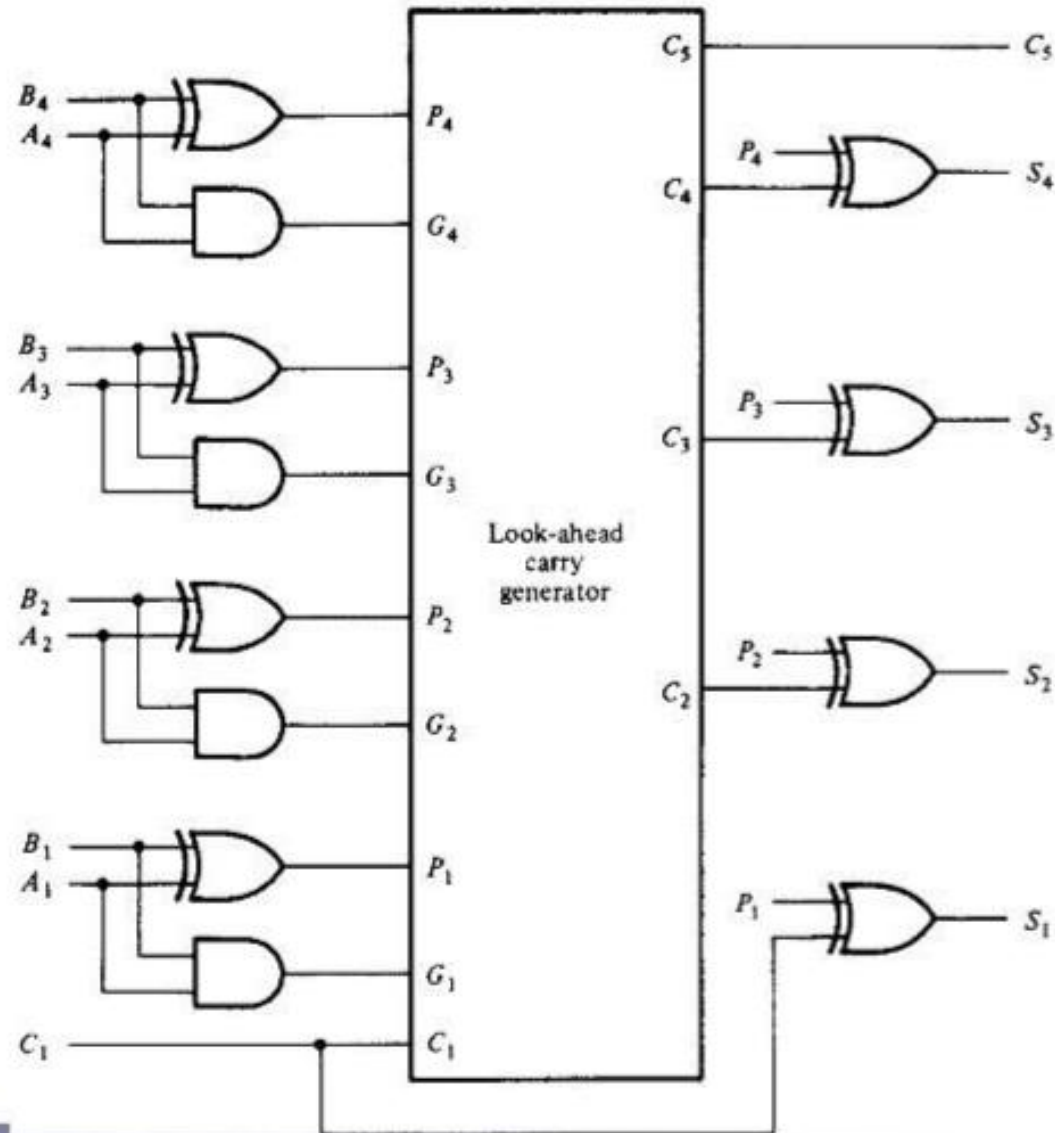
- So each function can be implemented with one level of AND Gates followed by an OR gate.
- Three Boolean function for C_2 , C_3 and C_4 are implemented in the look ahead carry generator.
- Note that C_4 does not have to wait for C_3 and C_2 to propagate; C_4 is propagated at the same time as C_2 and C_3 .



The construction of a 4 bit parallel adder with a look-ahead carry,

- Each sum output requires two exclusive OR Gate.
- The output of the first gate generate the P_i variable.
- And the AND gate generates the G_i .
- All the P 's and G 's are generated in two gate levels.
- The carries are propagated through a look-ahead carry generator and applied as inputs to the second exclusive OR gate.
- After the P and G signals settle into their steady state value, all output carries are generated after a delay of the two level of gates.

- Thus output S_2 through S_4 have equal propagation delay times.



Overview

- Iterative combinational circuits
- Binary adders
 - Half and full adders
 - Ripple carry
- **Binary subtraction**
- **Binary adder-subtractors**
 - Signed binary numbers
 - Signed binary addition and subtraction
 - Overflow
- **Other arithmetic functions**
 - Design by contraction

Unsigned Subtraction

■ Algorithm:

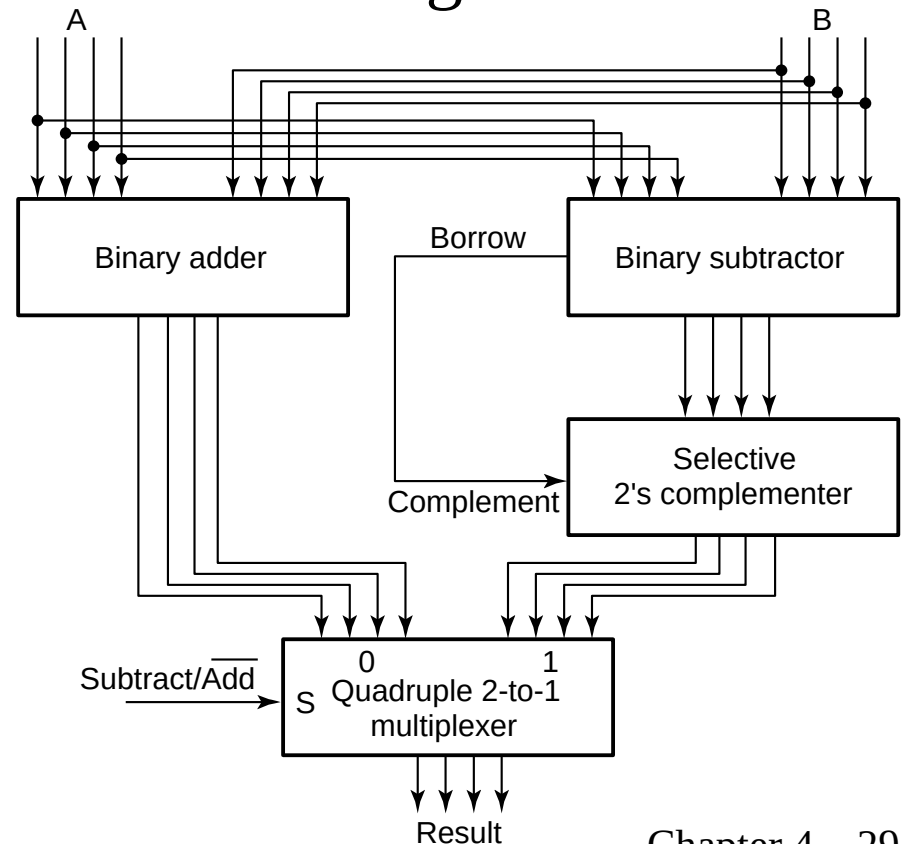
- Subtract the subtrahend N from the minuend M
- If no end borrow occurs, then $M \geq N$, and the result is a non-negative number and correct.
- If an end borrow occurs, then $N > M$ and the difference $M - N$ is subtracted from 2^n , and a minus sign is appended to the result.

■ Examples:

0	1
1001	0100
– 0111	– 0111
<u>0010</u>	<u>1101</u>
	10000
	– 1101
	(–) 0011

Unsigned Subtraction (continued)

- The subtraction, $2^n - N$, is taking the 2's complement of N
- To do both unsigned addition and unsigned subtraction requires:
- Quite complex!
- Goal: Shared simpler logic for both addition and subtraction
- Introduce complements as an approach



Complements

Complements are used in,

- Digital computers for simplifying the subtraction operation.
- There are two types of complements for each base- r system.
 - **r 's complement** ← Radix Complement
 - **$(r-1)$'s complement** ← Diminished Radix Complement
- When the value of the base is substituted the two types receive.
 - **2's and 1's complements for binary numbers, or**
 - **10's and 9's for decimal numbers.**

The r 's Complements

- Given a positive number N with an integer part of n digits, the r 's complement of N is defined as,

$$(r^n - N)$$

- For example,

- The 10's complement of $(52520)_{10}$ is $10^5 - 52520 = 47480$
 - Where n is number of digits in the Number.
- The 10's complement of $(0.3267)_{10}$ is $1 - 0.3267 = 0.6733$
 - No inter part so $10^n = 10^0 = 1$
- The 10's complement of $(25.639)_{10}$ is $10^2 - 25.639 = 74.631$

- For example,

- The 2's complement of $(101110)_2$ is $(2^6)_{10} - (101110)_2$
 $= (1000000 - 1011100)_2 = 010100$
- The 2's complement of $(0.0110)_2$ is $(1 - 0.0110)_2 = 0.1010$

Simple Observations

10's Complement of a number can be formed by leaving all least significant zeros unchanged, subtracting the first non zero least significant digit from 10, and then subtracting rest of the digits from 9.

The 2's complement can be formed by leaving all least significant zeros and the first nonzero digit unchanged, and then replacing 1 by 0's and 0's by 1's in all other higher significant digits.

Complements

- **Two complements:**
 - **Diminished Radix Complement of N**
 - $(r - 1)$'s complement for radix r
 - 1's complement for radix 2
 - Defined as $(r^n - 1) - N$
 - **Radix Complement**
 - r 's complement for radix r
 - 2's complement in binary
 - Defined as $r^n - N$
- **Subtraction is done by adding the complement of the subtrahend**
- **If the result is negative, takes its 2's complement**

Binary 2's Complement

- For $r = 2$, $N = 01110011_2$, $n = 8$ (8 digits), we have:

$$(r^n) = 256_{10} \text{ or } 100000000_2$$

- The 2's complement of 01110011 is then:

$$\begin{array}{r} 100000000 \\ - \underline{01110011} \\ 10001101 \end{array}$$

- Note the result is the 1's complement plus 1, a fact that can be used in designing hardware

Alternate 2's Complement Method

- **Given: an n -bit binary number, beginning at the least significant bit and proceeding upward:**
 - Copy all least significant 0's
 - Copy the first 1
 - Complement all bits thereafter.
- **2's Complement Example:**

10010100

- Copy underlined bits:

100

- and complement bits to the left:

01101100

The (r-1)'s Complement

- Given a positive number N with an integer part of n digits and a fraction part of m digits,
- the (r-1)'s complement of N is defined as,

$$(r^n - r^{-m} - N)$$

- For example,
 - **The 9's complement of $(52520)_{10}$ is $(10^5 - 1 - 52520) = 47479$**
 - **No fraction part, so $10^{-m} = 10^0 = 1$**
 - **The 9's complement of $(0.3267)_{10}$ is $(1 - 10^{-4} - 0.3267)$
 $= 0.9999 - 0.3267 = 0.6732$**
 - **No inter part so $10^n = 10^0 = 1$**
 - **The 9's complement of $(25.639)_{10}$ is $(10^2 - 10^{-3} - 25.639)$
 $= 99.999 - 25.639 = 74.360$**

■ ***The (r-1)'s Complement***

■ For example,

■ **The 1's complement of $(101110)_2$ is $(2^6-1) - (101100)_2$
 $= (111111 - 101100)_2 = 010011$**

■ **The 1's complement of $(0.0110)_2$ is $(1 - 2^{-4})10 - 0.0110)_2$
 $= (0.1111 - 0.0110) = 0.1001$**

We see from the above example that the 9's complement of a number is formed simply subtracting every digit from 9.

1's complement of a binary is formed by simply changing the bits from 1 to 0 and 0 to 1.

Binary 1's Complement

- For $r = 2$, $N = 01110011_2$, $n = 8$ (8 digits):

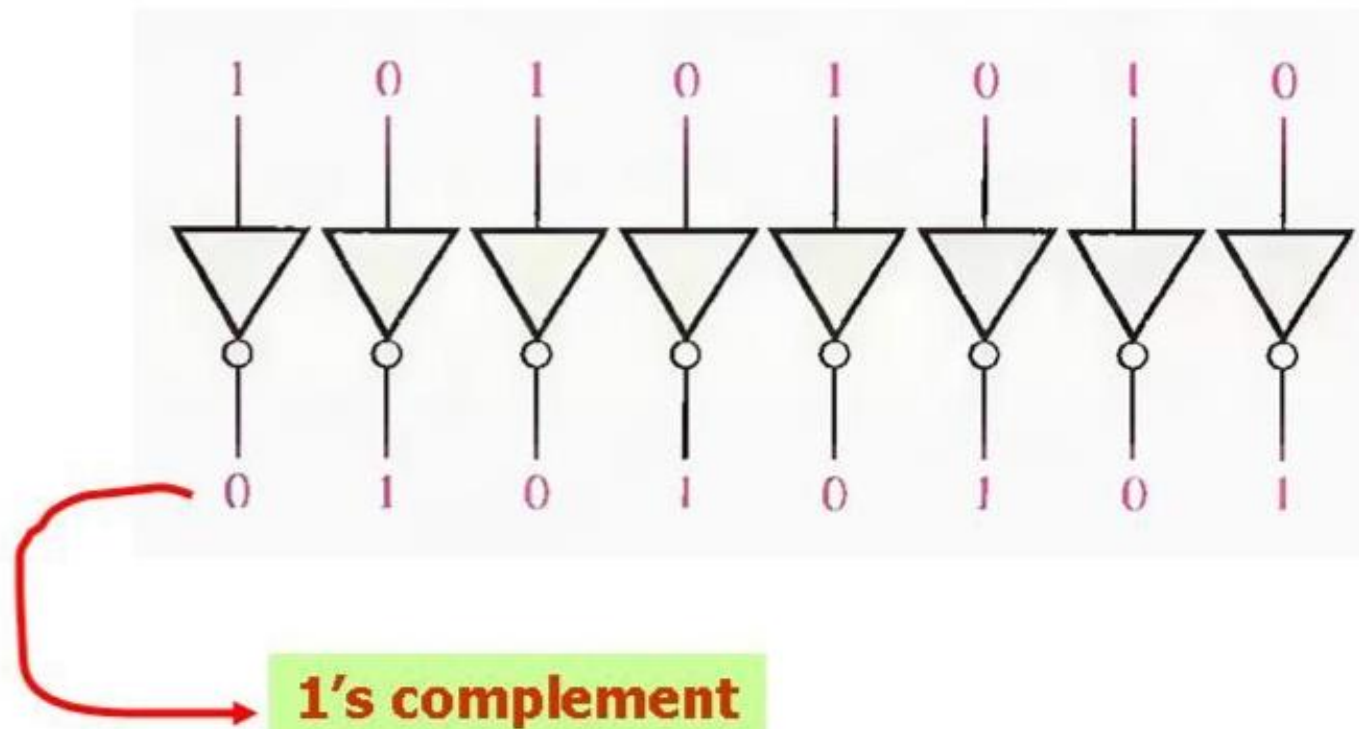
$$(r^n - 1) = 256 - 1 = 255_{10} \text{ or } 11111111_2$$

- The 1's complement of 01110011_2 is then:

$$\begin{array}{r} 11111111 \\ - 01110011 \\ \hline 10001100 \end{array}$$

- Since the $2^n - 1$ factor consists of all 1's and since $1 - 0 = 1$ and $1 - 1 = 0$, the one's complement is obtained by complementing each individual bit (bitwise NOT).

The simplest way to obtain the 1's complement of a binary number with a digital circuit is to use parallel inverters (NOT circuits),



Subtraction with 2's Complement

- For n-digit, unsigned numbers M and N, find $M - N$ in base 2:
 - Add the 2's complement of the subtrahend N to the minuend M:
$$M + (2^n - N) = M - N + 2^n$$
 - If $M \geq N$, the sum produces end carry r^n which is discarded; from above, $M - N$ remains.
 - If $M < N$, the sum does not produce an end carry and, from above, is equal to $2^n - (N - M)$, the 2's complement of $(N - M)$.
 - To obtain the result $-(N - M)$, take the 2's complement of the sum and place a $-$ to its left.

Subtraction with r's complement

Example: Using 10's comp. subtract 72532-3250.

$$M = 72532$$

$$N = 03250$$

$$\begin{array}{r} M = \phantom{10's \text{ complement of } N} 72532 \\ 10's \text{ complement of } N = + \underline{96750} \\ \hline \text{Sum} = \phantom{10's \text{ complement of } N} 169282 \end{array}$$

End Carry

Discard It

Answer = 69282

- **Example: Using 10's comp. subtract (3250 – 72532).**

$$M = 03250$$

$$N = 72532$$

$$\begin{array}{r} M = 03250 \\ 10\text{'s complement of } N = + \underline{27468} \\ \text{Sum} = 30718 \end{array}$$

No End Carry

$$\begin{aligned} \text{So Answer} &= - (10\text{'s complement of } 30718) \\ &= - 69282 \end{aligned}$$

Unsigned 2's Complement Subtraction Example 1

- Find $01010100_2 - 01000011_2$

$$\begin{array}{r} 01010100 \\ - 01000011 \\ \hline \end{array} \xrightarrow{\text{2's comp}} \begin{array}{r} 101010100 \\ + 10111101 \\ \hline 00010001 \end{array}$$

- The carry of 1 indicates that no correction of the result is required.

Unsigned 2's Complement Subtraction Example 2

- Find $01000011_2 - 01010100_2$

$$\begin{array}{r} 01000011 \\ - 01010100 \\ \hline \end{array} \xrightarrow{\text{2's comp}} \begin{array}{r} 0 \quad 01000011 \\ + 10101100 \\ \hline 11101111 \\ \hline 00010001 \end{array} \xrightarrow{\text{2's comp}}$$

- Result = $-(00010001)$

Practice

Given the two binary numbers $X = 1010100$ and $Y = 1000011$

Perform $X - Y$ and $Y - X$ using 2's complement

Subtraction with $(r-1)$'s Complement

Subtraction with $(r-1)$'s complement

Subtraction of two +ive number $(M-N)$ both of base r may be done as follows,

- Add the Minuend M to the $(r-1)$'s complement of the subtrahend N .
- Inspect the result obtained in step 1 for an end carry:
 - **If an end carry occurs, add 1 to least significant digit.**
 - **If an end carry does not occur, take the $(r-1)$'s comp. of the number obtained in step 1 and place a -ive sign in front of it.**

Example: Using 9's comp. subtract (72532 – 3250)

$$M = 72532$$

$$N = 03250$$

$$M = 72532$$

9's complement of N = +96749

$$\begin{array}{r} 69281 \\ +1 \\ \hline 69282 \end{array}$$

So Answer = 69282

- **Example: Using 9's comp. subtract (3250 - 72532)**

$$M = 03250$$

$$N = 72532$$

$$M = 03250$$

$$9\text{'s complement of } N = +27467$$

No End Carry

$$30717$$

$$\text{So Answer} = -(9\text{'s complement of } 30717)$$

$$= -69282$$

Example: Using 1's comp. subtract (1010100 - 1000100)

$$M = 1010100$$

$$N = 1000100$$

$$M = 1010100$$

1's complement of N = +0111011

$$\begin{array}{r} 0001111 \\ +1 \\ \hline 0010000 \end{array}$$

So Answer = 10000

Example: Using 1's comp. subtract (1000100 - 1010100)

$$M = 1000100$$

$$N = 1010100$$

$$M = 1000100$$

1's complement of N = +0101011

No End Carry

1101111



So Answer = -(1's complement of 1101111)

= - 10000

Overview

- Iterative combinational circuits
- Binary adders
 - Half and full adders
 - Ripple carry
- Binary subtraction
- **Binary adder-subtractors**
 - Signed binary numbers
 - Signed binary addition and subtraction
 - Overflow
- **Other arithmetic functions**
 - Design by contraction

Signed Integers

- Positive numbers and zero can be represented by unsigned n -digit, radix r numbers. We need a representation for negative numbers.
- To represent a sign (+ or –) we need exactly one more bit of information (1 binary digit gives $2^1 = 2$ elements which is exactly what is needed).
- Since computers use binary numbers, by convention, the most significant bit is interpreted as a sign bit:

$$s \ a_{n-2} \ \dots \ a_2 a_1 a_0$$

where:

$s = 0$ for Positive numbers

$s = 1$ for Negative numbers

and $a_i = 0$ or 1 represent the magnitude in some form.

Signed Integer Representations

- *Signed-Magnitude* – here the $n - 1$ digits are interpreted as a positive magnitude.
- *Signed-Complement* – here the digits are interpreted as the rest of the complement of the number. There are two possibilities here:
 - *Signed 1's Complement*
 - Uses 1's Complement Arithmetic
 - *Signed 2's Complement*
 - Uses 2's Complement Arithmetic

Signed Integer Representation Example

- $r = 2, n = 3$

Number	Sign -Mag.	1's Comp.	2's Comp.
+3	011	011	011
+2	010	010	010
+1	001	001	001
+0	000	000	000
−0	100	111	—
−1	101	110	111
−2	110	101	110
−3	111	100	101
−4	—	—	100

Signed-Magnitude Arithmetic

- **If the parity of the three signs is 0:**
 1. Add the magnitudes.
 2. Check for overflow (a carry out of the MSB)
 3. The sign of the result is the same as the sign of the first operand.
- **If the parity of the three signs is 1:**
 1. Subtract the second magnitude from the first.
 2. If a borrow occurs:
 - take the two's complement of result
 - and make the result sign the complement of the sign of the first operand.
 3. Overflow will never occur.

Sign-Magnitude Arithmetic Examples

- Example 1: 0010
 + 0101

- Example 2: 0010
 + 1101

Signed-Complement Arithmetic

- **Addition:**

1. Add the numbers including the sign bits, discarding a carry out of the sign bits (2's Complement), or using an end-around carry (1's Complement).
2. If the sign bits were the same for both numbers and the sign of the result is different, an overflow has occurred.
3. The sign of the result is computed in step 1.

- **Subtraction:**

Form the complement of the number you are subtracting and follow the rules for addition.

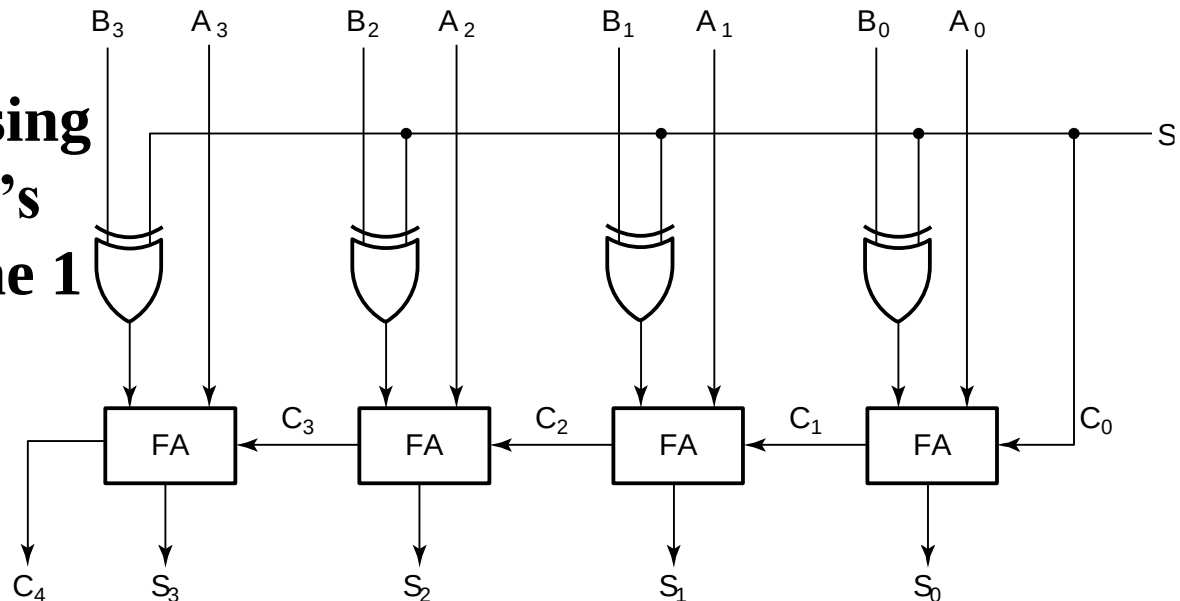
Signed 2's Complement Examples

- Example 1: 1101
 $+ \underline{0011}$

- Example 2: 1101
 $- \underline{0011}$

2's Complement Adder/Subtractor

- Subtraction can be done by addition of the 2's Complement.
 1. Complement each bit (1's Complement.)
 2. Add 1 to the result.
- The circuit shown computes $A + B$ and $A - B$:
- For $S = 1$, subtract, the 2's complement of B is formed by using XORs to form the 1's comp and adding the 1 applied to C_0 .
- For $S = 0$, add, B is passed through unchanged



Overflow Detection

- *Overflow* occurs if $n + 1$ bits are required to contain the result from an n -bit addition or subtraction
- Overflow can occur for:
 - Addition of two operands with the same sign
 - Subtraction of operands with different signs
- Signed number overflow cases with correct result sign

0	0	1	1
+ <u>0</u>	- <u>1</u>	- <u>0</u>	+ <u>1</u>
0	0	1	1
- Detection can be performed by examining the result signs which should match the signs of the top operand

Overflow Detection

- Signed number cases with carries C_n and C_{n-1} shown for correct result signs:

$$\begin{array}{r}
 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \\
 0 \quad 0 \quad 1 \quad 1 \\
 + \underline{0} \quad -\underline{1} \quad -\underline{0} \quad +\underline{1} \\
 0 \quad 0 \quad 1 \quad 1
 \end{array}$$

- Signed number cases with carries shown for erroneous result signs (indicating overflow):

$$\begin{array}{r}
 0 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 0 \\
 0 \quad 0 \quad 1 \quad 1 \\
 + \underline{0} \quad -\underline{1} \quad -\underline{0} \quad +\underline{1} \\
 1 \quad 1 \quad 0 \quad 0
 \end{array}$$

- Simplest way to implement overflow $V = C_n \oplus C_{n-1}$
- This works correctly only if 1's complement and the addition of the carry in of 1 is used to implement the complementation! Otherwise fails for $-10 \dots 0$

Overview

- Iterative combinational circuits
- Binary adders
 - Half and full adders
 - Ripple carry
- Binary subtraction
- Binary adder-subtractors
 - Signed binary numbers
 - Signed binary addition and subtraction
 - Overflow
- **Other arithmetic functions**
 - **Design by contraction**

Other Arithmetic Functions

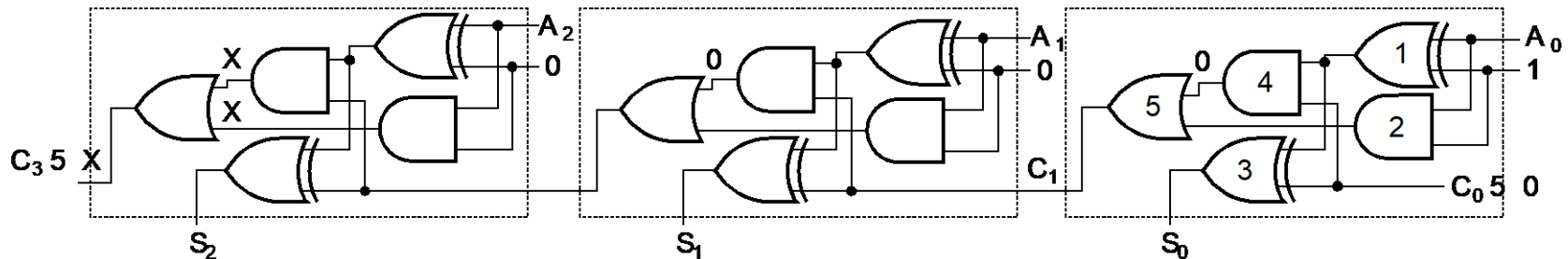
- Convenient to design the functional blocks by *contraction* - removal of redundancy from circuit to which input fixing has been applied
- Functions
 - Incrementing
 - Decrementing
 - Multiplication by Constant
 - Division by Constant
 - Zero Fill and Extension

Design by Contraction

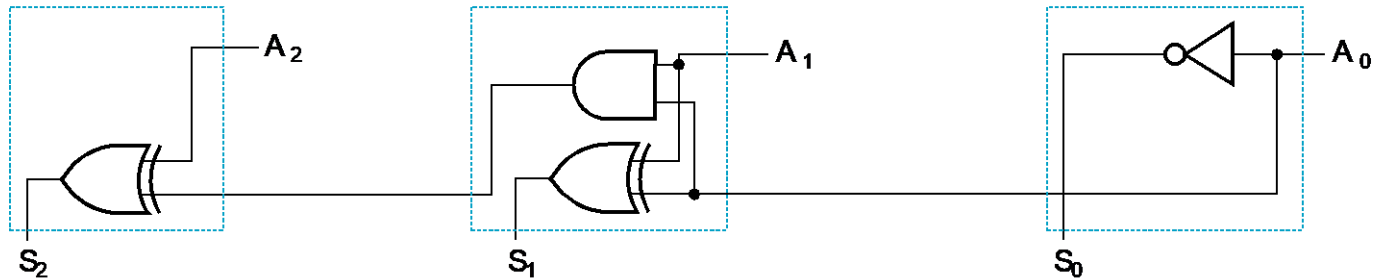
- **Contraction is a technique for simplifying the logic in a functional block to implement a different function**
 - **The new function must be realizable from the original function by applying rudimentary functions to its inputs**
 - **Contraction is treated here only for application of 0s and 1s (not for X and $\overline{X}$)**
 - **After application of 0s and 1s, equations or the logic diagram are simplified by using rules**

Design by Contraction Example

- Contraction of a ripple carry adder to incrementer for $n = 3$
 - Set B = 001



(a)



(b)

- The middle cell can be repeated to make an incrementer with $n > 3$.

Incrementing & Decrementing

■ *Incrementing*

- Adding a fixed value to an arithmetic variable
- Fixed value is often 1, called *counting (up)*
- Examples: $A + 1$, $B + 4$
- Functional block is called *incrementer*

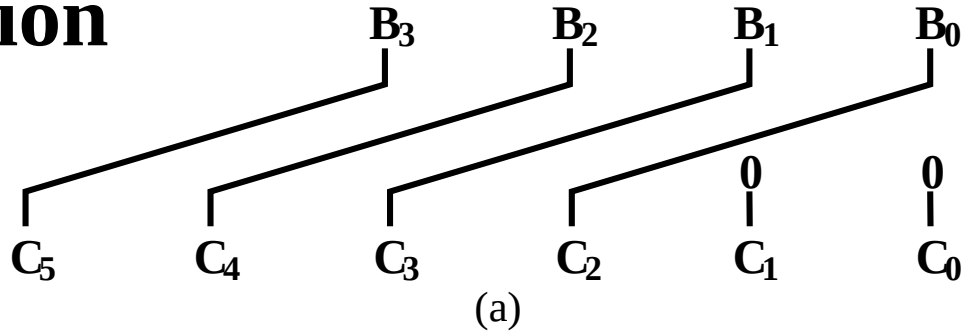
■ *Decrementing*

- Subtracting a fixed value from an arithmetic variable
- Fixed value is often 1, called *counting (down)*
- Examples: $A - 1$, $B - 4$
- Functional block is called *decrementer*

Multiplication/Division by 2^n

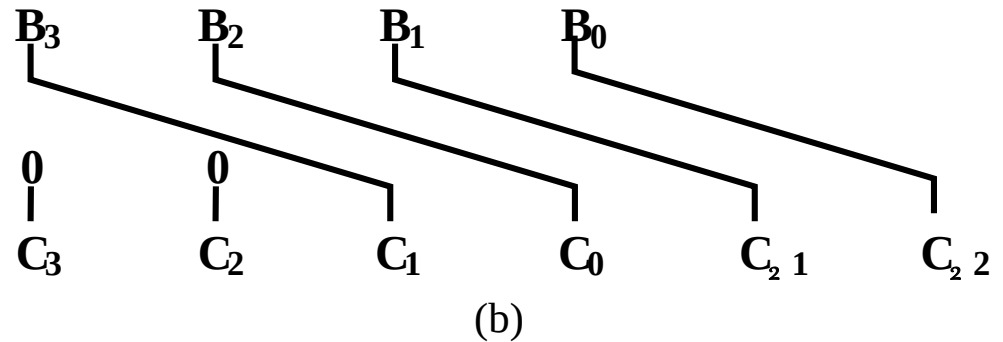
■ (a) Multiplication by 100

- Shift left by 2



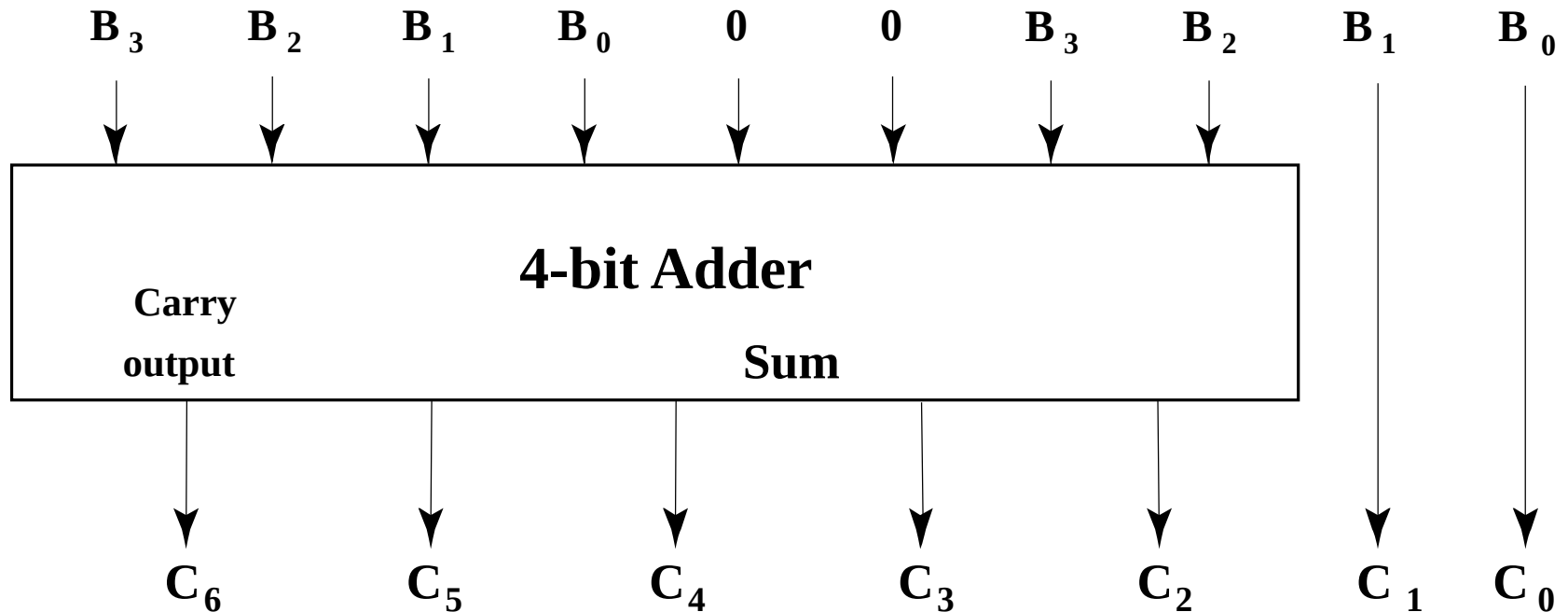
■ (b) Division by 100

- Shift right by 2
- Remainder preserved



Multiplication by a Constant

- Multiplication of $B(3:0)$ by 101



Zero Fill

- ***Zero fill*** - filling an m -bit operand with 0s to become an n -bit operand with $n > m$
- Filling usually is applied to the MSB end of the operand, but can also be done on the LSB end
- **Example: 11110101 filled to 16 bits**
 - MSB end: 0000000011110101
 - LSB end: 1111010100000000

Extension

- ***Extension*** - increase in the number of bits at the MSB end of an operand by using a complement representation

- Copies the MSB of the operand into the new positions
- Positive operand example - 01110101 extended to 16 bits:

0000000001110101

- Negative operand example - 11110101 extended to 16 bits:

1111111111110101